

Defense preparation — RideReady

Start here

The defense is 50% of the assignment grade. Be able to explain and modify the program without reading a script. Review the source and adapt the report to your own understanding. Confirm the course's rules on AI assistance before submission.

60-minute preparation plan

1. 10 minutes: run the demo and read Main, VehicleFactory, MotorcycleFactory and Vehicle.
2. 15 minutes: draw the Factory Method UML from memory and trace prepareRental().
3. 15 minutes: read AccessoryFactory and AccessoryRental; explain every dependency in the second diagram.
4. 10 minutes: run the tests and deliberately explain the mixed-family failure.
5. 10 minutes: rehearse the demonstration and answer the questions below aloud.

Five-minute demonstration script

- 0:00–0:30: “My topic is vehicle rentals. I create one vehicle with Factory Method, then a consistent accessory pair with Abstract Factory.”
- 0:30–1:30: Show Vehicle, VehicleFactory and both concrete creators. Follow prepareRental() into the overridden createVehicle().
- 1:30–2:30: Show the two accessory product interfaces and factories. Explain the helmet/lock and child-seat/alarm families.
- 2:30–3:15: Show AccessoryRental. Point out interface dependencies, final fields and construction validation. Explain Main's composition-root role.
- 3:15–4:00: Run the demo and tests. Explain both categories and the invalid-factory tests.
- 4:00–5:00: Use the UML to compare the patterns, state a tradeoff, and describe how to add a bicycle family.

Commands from the project directory:

```
powershell -ExecutionPolicy Bypass -File .\run.ps1
powershell -ExecutionPolicy Bypass -File .\run.ps1 -Mode test
```

Questions and concise answers

What is the Product in Part A? Vehicle. Motorcycle and Car are concrete products.

Where is the factory method? VehicleFactory.createVehicle(). Each concrete creator overrides it; prepareRental() calls it.

Why isn't this just a Simple Factory? A Simple Factory commonly centralizes selection in a method with a parameter and branches. Here subclasses override creation, and the inherited workflow uses dynamic dispatch.

Why have prepareRental()? It shows the purpose of the Creator: reusable behavior that consumes the product abstraction and delegates only creation.

What is the family in Part B? A safety product and a security product for one vehicle category. Motorcycle means Helmet plus DiscLock; car means ChildSeat plus CarAlarm.

Who is the Abstract Factory client? AccessoryRental. It receives AccessoryFactory and only uses SafetyEquipment and SecurityDevice.

Main constructs concrete factories. Is that wrong? No. An application needs a place to choose implementations. Main is that composition root. The reusable client does not make those choices.

Does the validation if violate the assignment? No. It validates matching family identifiers; it does not select or instantiate concrete products.

What is the distinction between the patterns? Factory Method delegates creation of a product to subclasses through an overridable method. Abstract Factory supplies a family through an injected factory object with multiple creation methods.

Can they be combined? Yes. Abstract Factory implementations can use factory methods internally. Here they demonstrate related creation decisions in separate parts.

How would you add a bicycle? Implement Bicycle and BicycleFactory. For accessories, add BICYCLE to VehicleCategory, implement both accessory interfaces, implement BicycleAccessoryFactory, and wire the pair in Main. Neither existing workflow nor client needs a type-selection branch.

How would you add insurance as a third family product? Add an Insurance interface and concrete family implementations, then add createInsurance() to AccessoryFactory and implement it in every factory. Update the client if it uses insurance. This is Abstract Factory's product-type extension cost.

Why return interfaces? Callers depend on contracts and can accept alternative implementations. The tests demonstrate substituting a custom safety product.

Why validate a factory that you wrote? A future or third-party implementation could return null or mix families. Validation protects the client invariant.

Do you guarantee a vehicle and accessory family match? Main supplies matching factories, but the model only validates consistency inside an accessory pair. Do not claim stronger compile-time safety.

Which Clean Code principles did you use? Meaningful names, polymorphism, small methods, encapsulation and abstraction, fail-fast validation without duplicated checks, and separation of composition from client behavior. Show the actual excerpts in the report.

Why no database or GUI? Neither is required; the console keeps creation patterns visible and allows a reliable live demonstration.

What did the tests establish? Both workflows and families work, invalid construction fails, and alternative implementations can be substituted. They do not establish production fitness or physical safety.

Practice modifications

1. Before editing, create a branch. Add Bicycle and BicycleFactory and predict the output.
2. Make a test factory return Helmet plus CarAlarm. Explain why construction throws.
3. Replace Helmet with another motorcycle SafetyEquipment implementation. Observe that AccessoryRental is unchanged.
4. Explain which files change when adding a family versus adding a product type.

Defense-day checklist

- Bring a working JDK 17 environment and open the project before class.

- Run both commands once; keep docs/demo-output.txt as a fallback record.
- Open both UML diagrams, the report and GitHub repository.
- Know your repository URL and the purpose of each package.
- Check your name and group, review the report, and follow the institution's disclosure rules.
- Confirm exact Week 2 and Week 3 dates on Moodle; the supplied assignment gives no calendar dates.
- Upload the report and repository link to Moodle before the specified deadline. GitHub upload does not submit to Moodle.